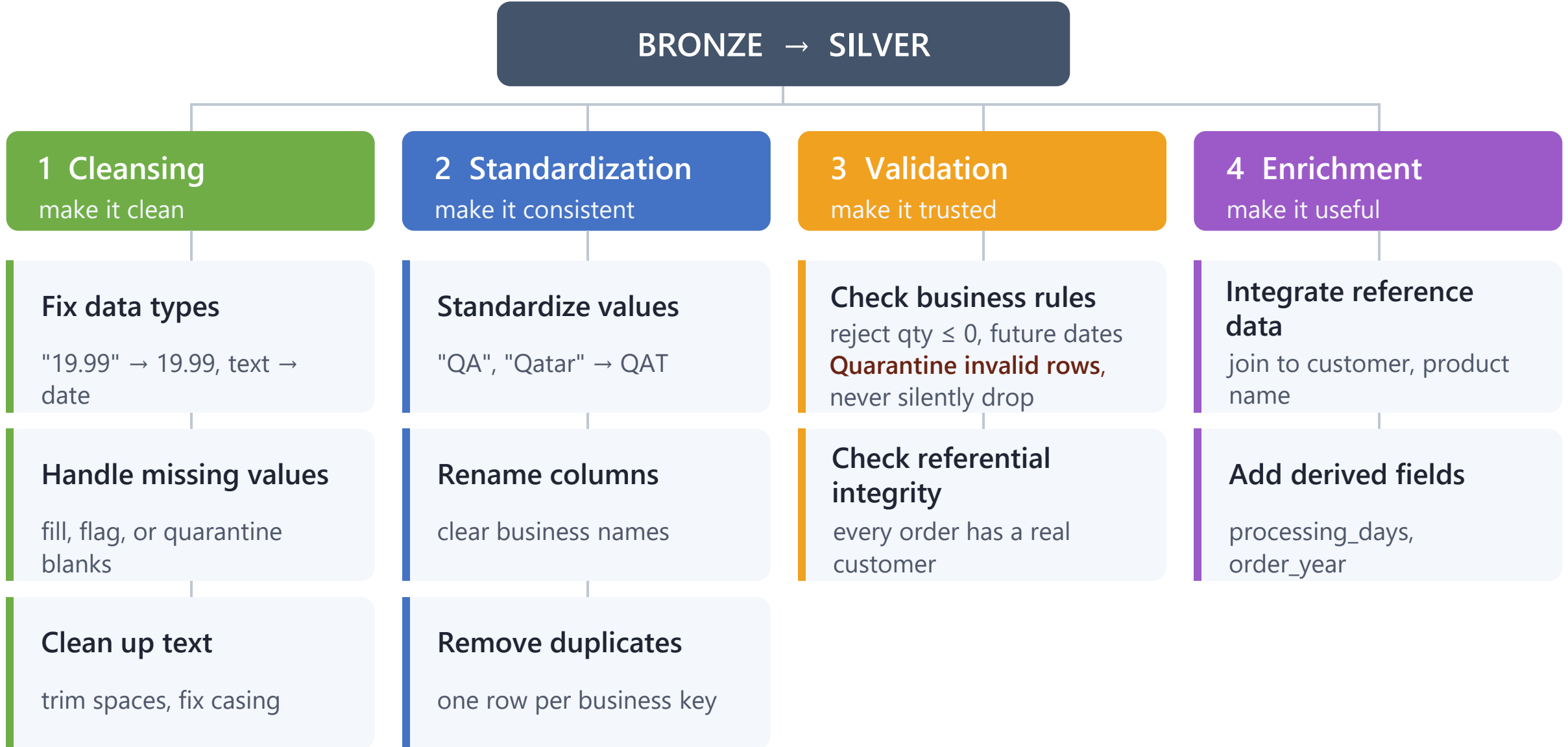


Silver Layer Transformations

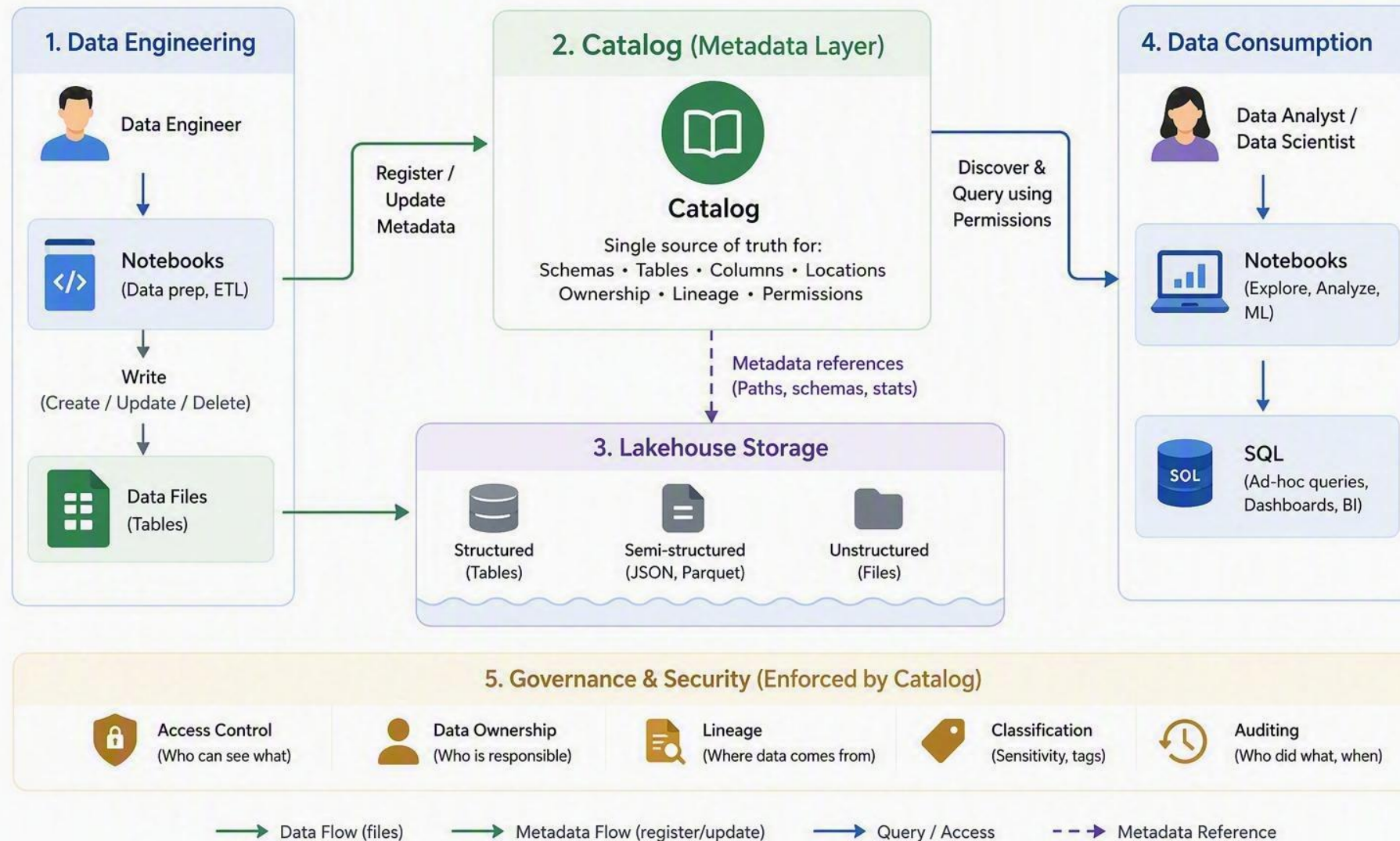


Silver Layer: Clean, Consistent, Trusted Data



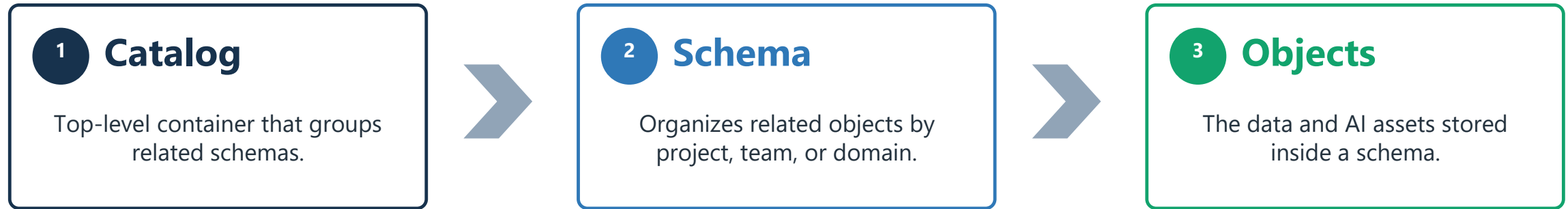
Lakehouse Data Flow and the Role of the Catalog

The catalog provides a single source of truth for data discovery, governance, and access.

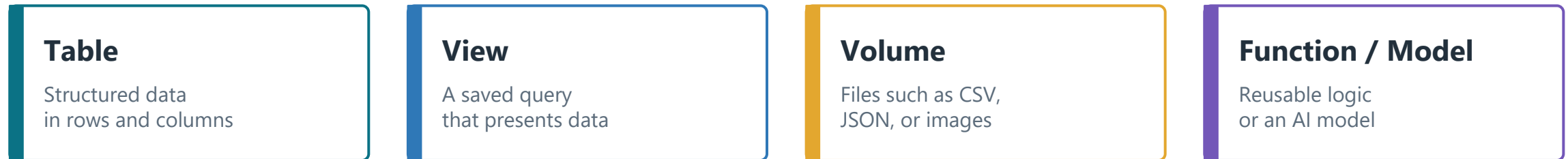


The Unity Catalog governance structure

A catalog contains schemas, and each schema contains data and AI objects.



Common objects inside a schema



Three-level name **catalog.schema.object**

Example **retail.sales.orders**

catalog schema table

What is the Silver Layer?

The layer where raw Bronze data becomes clean, consistent, trusted and useful

Silver is where every record is made typed, standardized, validated and enriched — while keeping one row per real-world entity/event.

What that means in practice

- Row-level detail is preserved — Silver is never aggregated
- Every column has a real type and a business-friendly name
- Invalid rows are quarantined, never silently dropped
- One reusable foundation for all Gold metrics

1 Cleansing make it clean

Fix data types, handle missing values, and cleanup text.

2 Standardization make it consistent

One name per column, one form per column values, one row per real-world entity.

3 Validation make it trusted

Enforce business rules, check referential integrity, and quarantine records that cannot be true.

4 Enrichment make it useful

Join to reference data, add derived fields, and stamp lineage and data-quality metadata.

BRONZE raw source data → SILVER clean + consistent + trusted + useful → GOLD metrics, aggregations, KPIs

Our Working Dataset

One retail dataset used across every transformation in this module

customers

customer_id, first_name, last_name, email, city, country, updated_date

orders

order_id, customer_id, order_date, status, updated_date

order_items

order_item_id, order_id, product_id, quantity, unit_price, updated_date

products

product_id, product_name, subcategory_code, unit_price, updated_date

product_category

subcategory_code, subcategory, category, updated_date

Bronze delivers all five tables as raw text → Silver returns them typed, deduplicated, validated and business-ready

1

Cleansing

Make it Clean

Data types | Missing values | Text cleanup

Data Type Enforcement

Cleansing | Cast raw text into correct business data types

What it is

- Bronze stores everything as text; silver assigns each column a real type
- Cast to DATE, DECIMAL, INT, BOOLEAN and enforce it on every load
- Schema enforcement quarantines records that will not cast

Why Silver needs it

- Dates must sort and support date arithmetic, not string comparison
- Money must be DECIMAL - never float - to avoid rounding drift
- A stable schema stops silent schema drift between loads

Common data quality issues

- order_date arriving as '2025-02-15' and '15/02/2025' in one file
- unit_price as text, so SUM() fails
- status as 'completed'/'Complete'/'COMPLETED' mixed across batches

▼ Impact downstream

String dates break time-series and partitioning; text money breaks every revenue metric downstream.

Problem → Detection → Transformation → Validation

Data Type Enforcement

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# What did the reader actually infer?
df.dtypes

# Which values refuse to become numbers?
bad = df[pd.to_numeric(df.unit_price,
                      errors='coerce').isna()]
```

SQL

```
-- Rows that will not cast to a date
SELECT order_id, order_date
FROM bronze.orders
WHERE TRY_CAST(order_date AS DATE) IS NULL
      AND order_date IS NOT NULL;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# errors='coerce' turns failures into NaT/NaN
df['order_date'] = pd.to_datetime(
    df.order_date, errors='coerce')
df['unit_price'] = pd.to_numeric(
    df.unit_price, errors='coerce')
df['customer_id'] = df.customer_id.astype('Int64')
```

SQL

```
SELECT
    CAST(customer_id AS INT)           AS customer_id,
    TRY_CAST(order_date AS DATE)       AS order_date,
    CAST(unit_price AS DECIMAL(10,2)) AS unit_price
FROM bronze.orders;
```

Data Type Enforcement

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Assert the final dtypes match the declared silver schema
- Count NaT/NULL created by the cast - a spike means a bad source format
- Re-check min/max: a date of 1900-01-01 signals a failed parse, not a real value

Classroom exercise

Load the raw orders file with no dtype hints, print `df.dtypes`, then cast every column and count how many values became NULL. Explain each one.

✓ Recommended practice

- Declare the schema explicitly instead of letting the reader infer it
- Use `TRY_CAST` / `errors='coerce'` so one bad row cannot kill the load
- Use `DECIMAL` for currency and store timestamps in UTC

✗ Common mistakes to avoid

- Silently coercing failures to NULL without counting them
- `FLOAT` for money - `0.1 + 0.2` will embarrass you in a finance report
- Letting each load infer its own types, producing schema drift

Safe Casting & Multi-Format Parsing

Cleansing | Convert text to types without failing the load

What it is

- `try_cast` / `try_strptime` return NULL instead of an error
- Clean before casting: trim spaces, strip ',' separators
- Parse dates with coalesce over each known format

Why Silver needs it

- One unparseable string must never abort an entire batch load
- A NULL result is a testable signal you can route to quarantine
- The same rule runs every night, so loads stay deterministic

Common data quality issues

- `unit_price` delivered as '1,500.00' - a plain CAST fails
- `order_date` as 2026-09-17, 2026/09/17 and 17-09-2026
- `updated_date` as free text with trailing spaces

▼ Impact downstream

A single unparseable value can fail the whole batch, or cast silently to zero and understate revenue.

Problem → Detection → Transformation → Validation

Safe Casting & Multi-Format Parsing

Detection & Implementation | Pandas and DuckDB SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# Which raw values refuse to become numbers?
px = df.unit_price.str.replace(',', ' ')
bad = df[pd.to_numeric(px, errors='coerce').isna()]
print(bad[["product_id", "unit_price"]])
```

SQL

```
-- Values that will not parse on this run
SELECT product_id, unit_price, updated_date
FROM bronze.products
WHERE try_cast(replace(trim(unit_price), ',', ' ')
               AS DECIMAL(10,2)) IS NULL;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Coerce, then keep the NULLs as evidence
df['unit_price'] = pd.to_numeric(
    df.unit_price.str.replace(',', ' '), errors='coerce')
df['order_date'] = pd.to_datetime(
    df.order_date, format='mixed', errors='coerce')
```

SQL

```
CREATE OR REPLACE TABLE memory.stg_products_cleaned AS
SELECT try_cast(trim(product_id) AS INTEGER) AS product_id,
       try_cast(replace(trim(unit_price), ',', ' ')
               AS DECIMAL(10,2)) AS unit_price,
       coalesce(try_strptime(trim(order_date), '%Y-%m-%d'),
               try_strptime(trim(order_date), '%d-%m-%Y'))
FROM bronze.products WHERE _load_date = DATE :run_date;
```

Handling Missing Values

Cleansing | Decide deliberately what every NULL means

What it is

- Detect true NULLs plus disguised ones: '', 'N/A', 'unknown', -1, 9999
- Choose per column: keep, default, enrich from reference data, or flag
- Silver preserves NULLs with meaning - it does not guess values

Why Silver needs it

- Downstream consumers need one consistent 'no value' representation
- Statistical imputation is a modelling choice and belongs in gold or ML
- A completeness metric per column becomes a pipeline health KPI

Common data quality issues

- country stored as '', 'N/A' and NULL in the same column
- city missing for 12% of customers
- unit_price missing on cancelled orders - legitimately absent

▼ Impact downstream

Mixed placeholders make COUNT and GROUP BY disagree, splitting one real category into four.

Problem → Detection → Transformation → Validation

Handling Missing Values

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# True nulls per column, worst first
df.isna().mean().sort_values(ascending=False)

# Disguised nulls hiding as text
placeholders = ['', 'N/A', 'NULL', 'unknown', '-']
df.country.isin(placeholders).sum()
```

SQL

```
SELECT
  COUNT(*) AS total,
  SUM(CASE WHEN country IS NULL THEN 1 ELSE 0 END)
    AS null_country,
  SUM(CASE WHEN TRIM(country) IN ('','N/A','unknown')
    THEN 1 ELSE 0 END) AS fake_null
FROM bronze.customers;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# 1) Unify every placeholder into a real NULL
df = df.replace(placeholders, pd.NA)

# 2) Flag before you fill - keep the evidence
df['country_missing'] = df.country.isna()
df['country'] = df.country.fillna('Unknown')
```

SQL

```
SELECT
  customer_id,
  COALESCE(NULLIF(TRIM(country),''), 'Unknown')
    AS country,
  CASE WHEN NULLIF(TRIM(country),'') IS NULL
    THEN 1 ELSE 0 END AS country_missing
FROM bronze.customers;
```

Handling Missing Values

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Recount nulls per column and compare against the agreed threshold
- Confirm no placeholder strings survive: `value_counts()` on the cleaned column
- Check the missing-flag total equals the original missing count

Classroom exercise

Profile every text column for disguised nulls, then produce a completeness percentage per column and defend which columns you left NULL.

✓ Recommended practice

- Convert every disguised placeholder to a real NULL first
- Add a boolean `*_missing` flag so the fact stays queryable
- Track completeness % per column as a data quality metric

✗ Common mistakes to avoid

- Mean/median imputation in silver - that is a modelling decision
- Dropping rows just because one column is missing
- Filling with 0, which is indistinguishable from a real zero

Text Cleanup

Cleansing | Trim, normalize casing, strip unwanted characters

What it is

- Remove leading, trailing and doubled whitespace
- Apply consistent casing rules per column (Title, UPPER, lower)
- Strip control characters, zero-width spaces and stray punctuation

Why Silver needs it

- Invisible whitespace silently breaks joins and GROUP BY
- 'doha', 'Doha' and 'DOHA' must become one city, not three
- Emails and codes need a canonical form before matching

Common data quality issues

- Names padded with spaces by a fixed-width export
- city recorded as 'doha', 'Doha ', 'DOHA'
- email as ' Ali@Shop.COM ' - case and space sensitive on join

▼ Impact downstream

A single trailing space creates a duplicate group in every report and a failed match in every join.

Problem → Detection → Transformation → Validation

Text Cleanup

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# Values whose length changes when trimmed
mask = df.city != df.city.str.strip()
print(mask.sum())

# How many casing variants of one city?
df.city.str.strip().str.lower().nunique()
```

SQL

```
-- Values carrying hidden whitespace
SELECT city, COUNT(*)
FROM bronze.customers
WHERE city <> TRIM(city)
GROUP BY city;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Chain: trim -> collapse inner spaces -> case
df['city'] = (df.city.str.strip()
             .str.replace(r'\s+', ' ', regex=True)
             .str.title())

# Emails are matched lowercase by convention
df['email'] = df.email.str.strip().str.lower()
```

SQL

```
SELECT
    INITCAP(TRIM(city))    AS city,
    LOWER(TRIM(email))     AS email,
    TRIM(first_name) || ' ' || TRIM(last_name)
    AS full_name
FROM bronze.customers;
```

Text Cleanup

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Assert no value differs from its trimmed form
- Compare distinct counts before and after - they should drop
- Spot-check that legitimate names like 'McDonald' survived the casing rule

✓ Recommended practice

- Trim first, then collapse inner whitespace, then apply casing
- Lowercase anything used as a join or match key, such as email
- Document the casing rule per column in the schema

Classroom exercise

Count distinct cities before and after cleanup. Explain the drop, then find one name your casing rule broke.

✗ Common mistakes to avoid

- Blind .title() on names - it produces 'Mcdonald' and 'O'Brien' errors
- Cleaning text after the join instead of before it
- Stripping characters that are meaningful, like the '-' in a product code

2

Standardization

Make it Consistent

Column naming | Value standardization | Deduplication

Column Naming & Schema Conventions

Standardization | One clear, business-friendly name per concept

What it is

- Rename cryptic source columns to business-readable names
- Apply one convention: snake_case, singular nouns, explicit units
- Drop technical columns that carry no business meaning

Why Silver needs it

- Silver is a shared, self-documenting contract for many consumers
- Consistent names make joins and gold-layer models predictable
- Good naming removes the need to read the pipeline code

Common data quality issues

- cst_id, CustomerID and customer_no all meaning customer_id
- amt with no unit - is it QAR, USD, or cents?
- Source audit columns like _src_file leaking into analytics

Impact downstream

Ambiguous names cause wrong joins and force every analyst to re-learn the same schema.

Problem → Detection → Transformation → Validation

Column Naming & Schema Conventions

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# Inspect what the source actually gave us
list(df.columns)

# Flag names breaking snake_case convention
bad = [c for c in df.columns
       if c != c.lower() or ' ' in c]
```

SQL

```
-- Compare source columns to the agreed contract
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'orders'
ORDER BY ordinal_position;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Explicit mapping beats clever string tricks
df = df.rename(columns={
    'cst_id':      'customer_id',
    'ord_dt':      'order_date',
    'amt':         'unit_price_usd',
}).drop(columns=['_src_file'])
```

SQL

```
SELECT
    cst_id AS customer_id,
    ord_dt AS order_date,
    amt AS unit_price_usd,
    status AS status
FROM bronze.orders; -- _src_file intentionally dropped
```

Column Naming & Schema Conventions

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Assert the final column list equals the documented silver schema exactly
- Confirm every name matches the convention with a regex check
- Verify join keys share identical names across all four tables

Classroom exercise

Given three source files using different names for the same concept, write one rename mapping and justify each chosen name to the class.

✓ Recommended practice

- Keep an explicit, reviewable rename mapping in version control
- Put units in the name: unit_price_usd, weight_kg
- Use the same key name in every table so joins are obvious

✗ Common mistakes to avoid

- Auto-generated renames that change when the source changes
- Abbreviations only the source system's authors understand
- Renaming a column's meaning without versioning the contract

Value Standardization

Standardization | One canonical representation per real-world value

What it is

- Map every variant of a value onto one agreed canonical form
- Keep the mapping in a reference table, not buried in code
- Route unmapped values to review instead of defaulting them

Why Silver needs it

- Reports must not split one category across several spellings
- Filters and GROUP BY only work on a controlled vocabulary
- Joins to dimension tables need matching, predictable keys

Common data quality issues

- country as 'QA', 'Qatar', 'qatar', 'QAT'
- country as 'USA', 'U.S.A.', 'US', 'United States'
- status as 'Shipped', 'SHIPPED', 'shp'

Impact downstream

Revenue by country splits across four spellings of one market, and filters silently miss rows.

Problem → Detection → Transformation → Validation

Value Standardization

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# The single most useful profiling line
df.country.value_counts(dropna=False)

# Which values are outside the allowed set?
allowed = {'QAT', 'ARE', 'Unknown'}
unmapped = set(df.country.dropna()) - allowed
```

SQL

```
-- Profile the real vocabulary of the column
SELECT country, COUNT(*) AS n
FROM bronze.customers
GROUP BY country
ORDER BY n DESC;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
country_map = {'qa':'QAT', 'qatar':'QAT', 'q.a.':'QAT',
               'ae':'ARE', 'uae':'ARE', 'u.a.e.':'ARE'}
key = df.country.astype(str).str.strip().str.lower()
# Unmapped values become 'Unknown', never silently dropped
df['country'] = key.map(country_map).fillna('Unknown')
```

SQL

```
SELECT customer_id,
       CASE LOWER(TRIM(country))
         WHEN 'qa'      THEN 'QAT'
         WHEN 'qatar'   THEN 'QAT'
         WHEN 'ae'      THEN 'ARE'
         WHEN 'uae'     THEN 'ARE'
         ELSE 'Unknown'
       END AS country
FROM bronze.customers;
```

Value Standardization

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Assert the distinct set equals the allowed vocabulary exactly
- Track the count landing in 'Unknown' - a rise means a new variant appeared
- Confirm row counts are unchanged: standardizing must never lose rows

Classroom exercise

Run `value_counts` on country and status, build the mapping, then prove no row landed outside the allowed vocabulary.

✓ Recommended practice

- Normalize case and whitespace before applying the mapping
- Store mappings in a reference table so business users can maintain them
- Alert when an unmapped value appears rather than defaulting quietly

✗ Common mistakes to avoid

- Hard-coding long mappings inline across several notebooks
- Assuming the source vocabulary is fixed - it will grow
- Mapping unknown values into a real category to make charts look tidy

Deduplication by Business Key

Standardization | One row per real-world entity, keeping the latest version

What it is

- Define the business key: order_id, or (order_id, product_id)
- Rank duplicates by a recency column and keep only the newest
- Makes reloads idempotent - re-running never doubles the data

Why Silver needs it

- Re-ingested batches and retries produce exact duplicate rows
- Source updates arrive as new rows, not in-place edits
- Duplicates inflate every SUM, COUNT and average downstream

Common data quality issues

- The same order_id ingested twice after a pipeline retry
- A customer record updated three times - which is current?
- Near-duplicates differing only in whitespace or casing

▼ Impact downstream

Revenue is overstated, customer counts inflate, and every KPI drifts in the same optimistic direction.

Problem → Detection → Transformation → Validation

Deduplication by Business Key

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# Exact duplicates across all columns
df.duplicated().sum()

# Business-key duplicates - the ones that matter
dups = df[df.duplicated('order_id', keep=False)]
dups.sort_values('order_id').head()
```

SQL

```
SELECT order_id, COUNT(*) AS n
FROM bronze.orders
GROUP BY order_id
HAVING COUNT(*) > 1
ORDER BY n DESC;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Keep the most recent row per business key.
# keep='last' is only correct because we sort first!
df = (df.sort_values('updated_date')
      .drop_duplicates('order_id', keep='last'))
```

SQL

```
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY order_id
        ORDER BY updated_date DESC) AS rn
    FROM bronze.orders)
SELECT * FROM ranked WHERE rn = 1;
```

Deduplication by Business Key

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Assert the business key is now unique: `COUNT(*) = COUNT(DISTINCT key)`
- Compare row counts before and after and log the number removed
- Re-run the pipeline on the same input - the output must be identical

Classroom exercise

Run your pipeline twice on the same input file and diff the outputs. If they differ, your transformation is not idempotent — find out why.

✓ Recommended practice

- Agree the business key with the business, not by guessing
- Always sort or rank by a recency column before dropping
- Log how many duplicates were removed as a quality metric

✗ Common mistakes to avoid

- `drop_duplicates()` with no subset - it only catches exact copies
- Deduplicating before text cleanup, so variants survive as distinct
- Assuming the source's primary key is unique in your landing zone

Idempotent Loads with MERGE

Standardization | Re-running a load must not duplicate or lose rows

What it is

- MERGE matches staged rows to Silver on the business key
- WHEN MATCHED THEN UPDATE refreshes the existing row in place
- WHEN NOT MATCHED THEN INSERT adds only the genuinely new key

Why Silver needs it

- Re-running yesterday's file leaves Silver in the same state
- CREATE TABLE IF NOT EXISTS keeps history; OR REPLACE destroys it
- Late corrections update the existing row instead of adding a twin

Common data quality issues

- The same order_id re-ingested after a pipeline retry
- A customer record arriving three times in one extract
- Full reloads that wipe already-published rows

▼ Impact downstream

Without an upsert key every retry inflates revenue, and every full reload loses corrections already applied.

Deduplicate → Stage → MERGE → Verify

Idempotent Loads with MERGE

Detection & Implementation | Latest row per key, then upsert

STEP 1 — DETECT THE ISSUE

Pandas

```
# How many versions of each key arrived?
n = df.groupby('subcategory_code').size()
print(n[n > 1])
```

SQL

```
-- Rank the versions of each business key
SELECT subcategory_code, updated_date,
       row_number() OVER (PARTITION BY subcategory_code
                          ORDER BY updated_date DESC) AS row_num
FROM memory.stg_category_cleaned;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Keep only the newest row per key, then upsert
stg = (df.sort_values('updated_date')
       .drop_duplicates('subcategory_code', keep='last'))
silver = (pd.concat([silver[~silver.subcategory_code
                        .isin(stg.subcategory_code)], stg]))
```

SQL

```
-- QUALIFY keeps row_number() = 1, then MERGE upserts
MERGE INTO silver.product_category AS t
USING memory.stg_category AS s
  ON t.subcategory_code = s.subcategory_code
WHEN MATCHED THEN UPDATE SET subcategory = s.subcategory,
                             category = s.category, updated_date = s.updated_date
WHEN NOT MATCHED THEN INSERT VALUES (s.*);
```

3

Validation

Make it Trusted

Business rules | Quarantine of invalid records

Business Rule Validation

Validation | Turn business assumptions into executable, enforced checks

What it is

- Encode domain constraints as testable rules on every load
- Cover ranges, formats, dates, and referential integrity
- Rules run as part of the pipeline, not as a one-off inspection

Why Silver needs it

- Clean formatting does not mean the value is plausible
- Orphan foreign keys silently drop rows in downstream joins
- Explicit rules become the documented contract for silver

Common data quality issues

- quantity of 0 or -5 on a transaction
- order_date in the future, or updated_date before order_date
- orders.customer_id with no matching row in customers

▼ Impact downstream

Impossible values distort averages, and orphan keys quietly shrink results in every inner join.

Problem → Detection → Transformation → Validation

Business Rule Validation

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
rules = {
    'qty_positive': df.quantity > 0,
    'date_not_future': df.order_date <= pd.Timestamp.today(),
    'total_ok': (df.quantity*df.unit_price).round(2)
                > 0,
}
{k: (~v).sum() for k, v in rules.items()} # failures
```

SQL

```
-- Orphan check: order_items with no parent order
SELECT t.order_item_id
FROM bronze.order_items t
LEFT JOIN silver.orders o ON t.order_id = o.order_id
WHERE o.order_id IS NULL;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Combine all rules into one pass/fail column
import numpy as np
df['is_valid'] = np.logical_and.reduce(
    list(rules.values()))
df['fail_reason'] = np.where(df.quantity <= 0,
    'qty_not_positive', None)
```

SQL

```
SELECT *,
    CASE WHEN quantity > 0
         AND updated_date <= CURRENT_DATE
         THEN TRUE ELSE FALSE END AS is_valid
FROM bronze.order_items;
```

Business Rule Validation

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Report pass and fail counts per rule on every single load
- Assert critical rules at 100% - fail the build if they are not
- Trend the failure rate over time to catch upstream regressions

✓ Recommended practice

- Write rules with the business owner, in their language
- Separate hard rules that block the load from soft ones that warn
- Record which rule failed, not just that validation failed

Classroom exercise

Write five business rules with a partner, then report pass/fail counts per rule. Which of your rules should block the load, and which should only warn?

✗ Common mistakes to avoid

- Rules hidden in notebook cells nobody runs in production
- Treating every violation as fatal - some are genuinely expected
- Validating only the format and never the business plausibility

Quarantining Invalid Records

Validation | Isolate bad rows without ever losing them

What it is

- Route failing rows to a quarantine table instead of dropping them
- Store the row, the failed rule, and the timestamp together
- Reprocess once the source is fixed - nothing is discarded

Why Silver needs it

- Silver must stay trustworthy without hiding upstream problems
- Quarantined rows are the evidence needed to fix the source
- One bad batch should never block the entire load

Common data quality issues

- A whole load failing because of five malformed rows
- Bad rows silently dropped, so totals never reconcile
- No record of what was rejected or why

▼ Impact downstream

Silent drops break reconciliation with the source and destroy confidence in the platform.

Problem → Detection → Transformation → Validation

Quarantining Invalid Records

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# Split, do not filter - keep both halves
valid  = df[df.is_valid]
invalid = df[~df.is_valid].copy()
print(len(valid), len(invalid))
assert len(valid) + len(invalid) == len(df)
```

SQL

```
-- How much are we rejecting, and why?
SELECT fail_reason, COUNT(*) AS n
FROM silver.order_items_quarantine
WHERE load_date = CURRENT_DATE
GROUP BY fail_reason;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
invalid['fail_reason'] = 'qty_not_positive'
invalid['quarantined_at'] = pd.Timestamp.now(tz='UTC')

valid.to_parquet('silver/order_items.parquet')
invalid.to_parquet('silver/order_items_quarantine.parquet')
```

SQL

```
INSERT INTO silver.order_items_quarantine
SELECT *, 'qty_not_positive' AS fail_reason,
        CURRENT_TIMESTAMP AS quarantined_at
FROM staged WHERE NOT is_valid;

INSERT INTO silver.order_items
SELECT * FROM staged WHERE is_valid;
```

Quarantining Invalid Records

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Reconcile: valid + quarantined must equal the input row count
- Alert when the quarantine rate exceeds an agreed threshold
- Confirm reprocessed rows leave quarantine once the source is fixed

Classroom exercise

Reconcile input rows against valid plus quarantined rows. Then fix one source record, replay it, and watch it graduate out of quarantine.

✓ Recommended practice

- Always record the reason and timestamp alongside the row
- Monitor quarantine volume as a first-class pipeline health metric
- Give the source system owner visibility of what was rejected

✗ Common mistakes to avoid

- Deleting invalid rows - you lose the evidence and the audit trail
- A quarantine table nobody ever reviews - it becomes a landfill
- Failing the entire load because a small fraction of rows are bad

Quarantine Refresh & Re-admission

Validation | Rejected rows are re-checked on every run

STEP 1 — DETECT THE ISSUE

Pandas

```
# Split the staged batch on the business rules
ok = stg[stg.product_id.notna() & (stg.unit_price >= 0)]
bad = stg.drop(ok.index)
print("rejected:", len(bad))
```

SQL

```
-- What is sitting in quarantine, and why?
SELECT rejection_reason, count(*) AS rows
FROM quarantine.products
GROUP BY rejection_reason
ORDER BY rows DESC;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Drop old rejects for keys seen again this run
q = q[~q.product_id.isin(stg.product_id)]
bad = bad.assign(rejection_reason='Invalid unit_price',
                 _load_date=run_date)
q = pd.concat([q, bad])
```

SQL

```
-- Clear this run's keys, then re-insert the failures
DELETE FROM quarantine.products WHERE product_id IN
    (SELECT product_id FROM memory.stg_products);
INSERT INTO quarantine.products SELECT *,
    CASE WHEN product_id IS NULL THEN 'Invalid product_id'
         WHEN unit_price < 0 THEN 'unit_price must be >= 0'
         ELSE 'Invalid product' END, DATE :run_date
FROM memory.stg_products WHERE unit_price < 0;
```

4

Enrichment

Make it useful

Integrate reference data • Add derived fields

Integrate Reference Data

Enrichment | Attach descriptive attributes from reference tables to each fact row

What it is

- Join fact tables to their reference (dimension) tables to bring in descriptive attributes.
- An orders row holds customer_id; after integration it also carries country and customer name.
- A order_items row holds product_id; after integration it also carries product_name and category.

Why Silver needs it

- A Silver table should be readable on its own — analysts should not need four joins to answer a question.
- The join itself is a final integrity check: an unmatched key is a real data problem.
- Conforms every consumer to one shared definition of "customer" and "product".

Common data quality issues

- Orphan foreign keys — an order pointing at a customer_id that does not exist
- Duplicated reference rows causing fan-out: 1 order becomes 3 rows
- Key mismatch from casing or padding: "C001" vs " c001 "
- Late-arriving dimensions — the product row loads after the transaction

▼ Impact downstream

Inner joins silently delete facts, so revenue quietly disappears; fan-out double-counts it. Both produce dashboards that look fine and are wrong.

Problem → Detection → Transformation → Validation

Integrate Reference Data

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# 1) Orphan keys: fact rows with no matching customer
orphans = orders.loc[
    ~orders.customer_id.isin(customers.customer_id)]
print("orphan orders:", len(orphans))

# 2) Is the reference table unique on its key?
# If not, a left join will multiply fact rows
dups = customers.customer_id.duplicated().sum()
print("duplicate customer keys:", dups)
```

SQL

```
-- 1) Orphan keys: anti-join pattern
SELECT o.order_id, o.customer_id
FROM   silver.orders o
LEFT JOIN silver.customers c
      ON c.customer_id = o.customer_id
WHERE  c.customer_id IS NULL;

-- 2) Reference table must be unique on its key
SELECT customer_id, COUNT(*) AS n
FROM   silver.customers
GROUP BY customer_id HAVING COUNT(*) > 1;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# LEFT join keeps every fact row, matched or not
# validate="m:1" raises if customers is not unique
orders_enr = orders.merge(
    customers[["customer_id", "country", "last_name"]],
    on="customer_id", how="left",
    validate="m:1", indicator=True)

# Make an unmatched key visible, never invisible
orders_enr["country"] = (
    orders_enr["country"].fillna("UNKNOWN"))
```

SQL

```
-- LEFT JOIN preserves every order row
CREATE OR REPLACE TABLE silver.orders_enriched AS
SELECT o.*,
      -- label the gap instead of hiding it
      COALESCE(c.country, 'UNKNOWN') AS country,
      c.last_name AS customer_last_name
FROM   silver.orders o
LEFT JOIN silver.customers c
      ON c.customer_id = o.customer_id;
```

Integrate Reference Data

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Row count unchanged: `len(orders_enr) == len(orders)` — a join must never change the grain.
- Unmatched rows: `(orders_enr._merge == "left_only").sum()` should equal the orphan count.
- No unexpected nulls in the new columns beyond those known orphans.
- Spot-check three orders by hand against the customers table.

Classroom exercise

Left-join `order_items` to `products`, then report how many `order_items` reference a `product_id` that is missing or inactive. Confirm the row count is identical before and after the join.

✓ Recommended practice

- Always LEFT join from fact to reference — the fact table sets the grain.
- Use `validate="m:1"` in Pandas (or a uniqueness test in SQL) so fan-out fails loudly.
- Keep the original key alongside the new descriptive column.
- Build the reference table first: deduplicate and validate it before anything joins to it.

✗ Common mistakes to avoid

- INNER JOIN on a fact table — it deletes rows without telling you.
- Joining on raw text keys before standardization has trimmed and cased them.
- Dropping the foreign key once you have the name — you lose the ability to re-join.
- Copying the whole reference table in; take only the columns consumers need.

Add Derived Fields

Enrichment | Compute business values once, centrally, at row level

What it is

- Add new columns calculated from existing columns in the same row.
- $\text{processing_days} = \text{updated_date} - \text{order_date}$; $\text{line_total} = \text{quantity} \times \text{unit_price}$; order_year .
- Also used for flags and buckets: is_first_order , age_band , is_delayed .

Why Silver needs it

- Encodes business logic once, in the pipeline, instead of separately in every report.
- Guarantees every consumer computes "processing_days" the same way.
- Moves expensive, repeated calculations out of dashboards and into a single scheduled job.

Common data quality issues

- Null inputs producing silent nulls or, worse, silent zeros
- Division by zero and negative durations from out-of-order timestamps
- Non-deterministic logic using `CURRENT_DATE`, so a re-run gives different answers
- Rounding and unit drift — cents versus dollars, days versus

▼ Impact downstream

Every team defines the metric slightly differently, three dashboards disagree, and the meeting turns into an argument about whose number is right.

Problem → Detection → Transformation → Validation

Add Derived Fields

Detection & Implementation | Pandas and SQL side by side

STEP 1 — DETECT THE ISSUE

Pandas

```
# Are the inputs safe to compute with?
print(orders[["order_date", "updated_date"]]
      .isna().sum())

# Impossible values: delivery before the order
bad = orders[orders.updated_date < orders.order_date]
print("negative durations:", len(bad))

# Zero denominators before any division
print("zero qty:", (order_items.quantity==0).sum())
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# Duration in whole days; stays NULL if input is NULL
orders["processing_days"] = (
    orders.updated_date - orders.order_date).dt.days

# Calendar attribute for easy period filtering
orders["order_year"] = orders.order_date.dt.year
# Null-safe flag: unknown date -> unknown flag
orders["is_delayed"] = (
    orders.processing_days.gt(7)
    .mask(orders.processing_days.isna()))
```

SQL

```
-- Null inputs and impossible orderings
SELECT
    COUNT(*) AS total,
    SUM(CASE WHEN updated_date IS NULL
             THEN 1 ELSE 0 END) AS missing_delivery,
    SUM(CASE WHEN updated_date < order_date
             THEN 1 ELSE 0 END) AS negative_days
FROM silver.orders_validated;
```

SQL

```
CREATE OR REPLACE TABLE silver.orders_enriched AS
SELECT o.*,
    -- NULL in, NULL out: never fake a value
    DATEDIFF(DAY, o.order_date, o.updated_date)
        AS processing_days,
    EXTRACT(YEAR FROM o.order_date) AS order_year,
    CASE WHEN DATEDIFF(DAY, o.order_date, o.updated_date) > 7
         THEN TRUE ELSE FALSE END AS is_delayed
FROM   silver.orders_validated o;
```

Add Derived Fields

Validation & Best Practices | Prove it worked, then keep it working

✓ How to verify it worked

- Recompute a five-row sample by hand and compare it against the column.
- Range-check: `processing_days` is never negative; above 365 deserves a look.
- Null count in the new column equals the null count of its inputs.
- Run the job twice on the same input — results must be identical.

Classroom exercise

Add `line_total = quantity × unit_price` to `order_items`, then check that no `line_total` is negative and that every `order_id` has at least one item row. Report how many orders disagree and explain what a mismatch means.

✓ Recommended practice

- Use only deterministic functions of the row — same input, same output, every run.
- Let NULL propagate rather than substituting 0, which quietly corrupts averages.
- Document each derived column: formula, unit, and owner.
- Keep the source columns so the calculation can always be audited or corrected.

✗ Common mistakes to avoid

- Aggregating — SUM, AVG and COUNT belong in Gold, not in a Silver derived column.
- Using `CURRENT_DATE` inside a stored column; store the event date and calculate age at read time.
- Burying business rules in a dashboard formula instead of the pipeline.
- Filling a failed calculation with 0 — it is indistinguishable from a genuine zero.

Integrated Sales Detail Table

Enrichment | One row per validated order line, ready for Gold

What it is

- sales_detail joins order_items to its order and its customer
- Carries product_name, category, customer_name, city and country
- Still row-level detail - it is a join, never an aggregation

Why Silver needs it

- Gold builds every KPI from one agreed, conformed table
- Dimensions resolve first, then orders, then order items
- INNER JOIN guarantees only referentially valid lines survive

Common data quality issues

- Fact tables built before dimensions are deduplicated
- Fan-out from a duplicated reference row inflates revenue
- Each dashboard re-implements the same join differently

Impact downstream

One conformed table replaces a dozen near-identical joins, so every downstream report agrees by construction.

Validate → Enrich → Conform → Hand off to Gold

Integrated Sales Detail Table

Detection & Implementation | Building silver.sales_detail

STEP 1 — DETECT THE ISSUE

Pandas

```
# Orphans vanish silently from an inner join
orphans = items[~items.order_id.isin(orders.order_id)]
print("orphan lines:", len(orphans))
```

SQL

```
-- Lines pointing at an order that does not exist
SELECT count(*) FROM silver.order_items i
LEFT JOIN silver.orders o ON i.order_id = o.order_id
WHERE o.order_id IS NULL;
```

STEP 2 — APPLY THE TRANSFORMATION

Pandas

```
# validate="m:1" raises if a dimension is not unique
sales = (items.merge(orders, on="order_id", validate="m:1")
        .merge(customers, on="customer_id",
               validate="m:1"))
sales["line_total"] = sales.quantity * sales.unit_price
```

SQL

```
CREATE OR REPLACE TABLE silver.sales_detail AS
SELECT i.order_id, o.order_date, o.status,
       c.customer_name, c.city, c.country,
       i.product_name, i.category,
       i.quantity, i.unit_price, i.line_total
FROM silver.order_items i
JOIN silver.orders o    ON i.order_id = o.order_id
JOIN silver.customers c ON o.customer_id = c.customer_id;
```

Silver Quality Scorecard

Validation | Counts that must all be zero after every load

STEP 1 — MEASURE EVERY RULE

Pandas

```
# One number per rule, on every single run
checks = {
    "dup_customers": customers.customer_id.duplicated().sum(),
    "bad_prices": (products.unit_price < 0).sum(),
    "non_positive_qty": (items.quantity <= 0).sum()}
```

SQL

```
-- Duplicates on the business key
SELECT count(*) AS duplicate_customers FROM (
    SELECT customer_id FROM silver.customers
    GROUP BY customer_id HAVING count(*) > 1);
```

STEP 2 — FAIL THE RUN, LOUDLY

Pandas

```
# Never let a silent failure reach Gold
failed = {k: v for k, v in checks.items() if v}
assert not failed, f"Silver DQ failed: {failed}"
```

SQL

```
-- Referential integrity across the Silver model
SELECT
    (SELECT count(*) FROM silver.orders o
     LEFT JOIN silver.customers c USING (customer_id)
     WHERE c.customer_id IS NULL) AS orders_no_customer,
    (SELECT count(*) FROM silver.order_items
     WHERE quantity <= 0) AS non_positive_qty;
```

Silver Layer: The Complete Checklist

From raw Bronze text to clean, consistent, trusted and useful Silver tables

1 Cleansing

make it clean

- ✓ Enforce data types
- ✓ Handle missing values
- ✓ Clean up text
- ✓ Cast safely with `try_cast`

2 Standardization

make it consistent

- ✓ Standardize column names
- ✓ Standardize values
- ✓ Deduplicate on business key
- ✓ Load idempotently with `MERGE`

3 Validation

make it trusted

- ✓ Apply business rules
- ✓ Check referential integrity
- ✓ Quarantine invalid records
- ✓ Re-admit fixed quarantine rows

4 Enrichment

make it useful

- ✓ Integrate reference data
- ✓ Add derived fields
- ✓ Build the `sales_detail` table

BRONZE raw source data → **SILVER** clean + consistent + trusted + useful → **GOLD** metrics, aggregations, KPIs

Silver stays detailed — never aggregated. Aggregation is the Gold layer's job.